Alembic

Schema Migrations for SQLAlchemy

Burak Bayındır · 150200340

YZV 322E — Applied Data Engineering · Spring 2026

ITU · Department of AI and Data Engineering

May 2026

What Is Alembic?

> "*A lightweight database migration tool for usage with the SQLAlchemy Database Toolkit for Python.*" — official site

- **First released:** 2010
- **Current version:** 1.13.x (latest 1.18 line, Feb 2026)
- **Author:** Mike Bayer (creator of SQLAlchemy)
- **Maintained by:** the SQLAlchemy team (zzzeek, CaselIT)
- **License:** MIT · Pure Python · supports Python 3.10+
- **Works with any DB SQLAlchemy supports:** PostgreSQL, MySQL, SQLite, Oracle, MS SQL

What Problem Does It Solve?

Schemas evolve. Production databases cannot just be re-created.

Manual `ALTER TABLE` scripts cause **drift** between dev / staging / prod, no rollback path, no audit trail.

Alembic provides:

- ✅ **Versioned migrations** as Python files in git
- ✅ Symmetric **upgrade / downgrade**
- ✅ **Autogenerate** from SQLAlchemy ORM models
- ✅ Branches and merges for team workflows
- ✅ `alembic_version` table tracks current state inside the DB

If prod is at schema X and dev is at Y, the answer is just `alembic upgrade head`.

Compared to Alternatives

	Alembic (Python)	Flyway (Java)	Liquibase (Java)
Migration format	Python files	SQL files	XML / YAML / JSON / SQL
Autogenerate from ORM	✅ yes (`Base.metadata`)	❌ no	❌ no
Rollback / undo	✅ free, symmetric	⚠️ paid edition	✅ declarative
Runtime	Python only	JVM required	JVM required
License	MIT (free)	Apache 2.0 + paid editions	Apache 2.0 + paid editions

Alembic's edge: ORM-aware autogenerate, plain Python migrations, no JVM.

Trade-off: tied to the SQLAlchemy / Python ecosystem.

Course Connection — Where It Fits

Belongs to Week 3: PostgreSQL / pgAdmin track of YZV 322E.

Tool	Role
PostgreSQL	OLTP / warehouse store
pgAdmin	GUI **inspection** of state
Alembic	**Change-control** of state (in git)
Airflow	DAG runs `alembic upgrade head` as first task before downstream ETL

- Same code runs in dev / CI / prod (matches the course's Docker-everywhere philosophy)
- Schema lives in the same git history as the application code
- pgAdmin *reads* the DB, Alembic *writes* the DB — together they cover the full DBA workflow

Setup with Docker

Architecture: `app` (python:3.12-slim + alembic + sqlalchemy + psycopg2) ⇄ `db` (postgres:16-alpine + `pg_isready` healthcheck) on a shared compose network. Both read `DATABASE_URL` from `.env`.

One-time setup — four commands, copy-pasteable:

```
git clone https://github.com/Budi4/alembic-presentation.git
cd alembic-presentation
cp .env.example .env
docker compose up -d --build
```

Healthcheck: `depends_on: { db: { condition: service_healthy } }` blocks `app` from starting until Postgres is ready. Avoids the database-not-ready race.

Project Layout & Configuration

```
alembic-presentation/
├── app/
│   ├── db.py
│   ├── models.py
│   └── models_with_tag.py
├── alembic/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       ├── 0001_create_users…
│       ├── 0002_add_email…
│       └── 0003_create_posts…
├── alembic.ini
├── docker-compose.yml
├── Dockerfile
├── demo.sh
└── requirements.txt
```

Three files do all the work:

- **`alembic.ini`** — points to `alembic/`; `sqlalchemy.url` left **blank**
- **`alembic/env.py`** — reads `DATABASE_URL` from env, sets `target_metadata = Base.metadata`, enables `compare_type=True`
- **`app/models.py`** — SQLAlchemy ORM models (`User`, `Post`)

Why this matters: same code runs in dev / CI / prod — only the env var changes.

Live Demo — Part A: Apply Migrations

```
$ docker compose exec app alembic current
(empty — no current revision)

$ docker compose exec app alembic history
0001 → 0002 → 0003 (head)

$ docker compose exec app alembic upgrade head
INFO  Running upgrade  -> 0001, create users table
INFO  Running upgrade 0001 -> 0002, add email column to users
INFO  Running upgrade 0002 -> 0003, create posts table

$ docker compose exec db psql -U alembic_demo -d alembic_demo -c "\dt"
 public | alembic_version | table | alembic_demo
 public | posts           | table | alembic_demo
 public | users           | table | alembic_demo
```

Each migration is a small Python file with `op.create_table(...)`, `op.add_column(...)`, etc. — reviewable in git, reversible, idempotent.

Live Demo — Part B: Autogenerate & Rollback

1. Edit `app/models.py`: add a `Tag` model.

2. Let Alembic write the migration for you:

```
$ docker compose exec app alembic revision --autogenerate -m "add tags table"
INFO  [alembic.autogenerate.compare] Detected added table 'tags'
  Generating /workspace/alembic/versions/<hash>_add_tags_table.py … done
```

3. Apply and roll back:

```
$ docker compose exec app alembic upgrade head
INFO  Running upgrade 0003 -> <hash>, add tags table

$ docker compose exec app alembic downgrade -1
INFO  Running downgrade <hash> -> 0003, add tags table
```

Every step is reversible, in git, and reproducible.

References

- **Official docs:** https://alembic.sqlalchemy.org/en/latest/
- **Tutorial:** https://alembic.sqlalchemy.org/en/latest/tutorial.html
- **Autogenerate:** https://alembic.sqlalchemy.org/en/latest/autogenerate.html
- **GitHub:** https://github.com/sqlalchemy/alembic
- **PyPI:** https://pypi.org/project/alembic/

Compared tools

- Flyway: https://documentation.red-gate.com/fd
- Liquibase: https://www.liquibase.com/community

Demo repository (this presentation)

`github.com/Budi4/alembic-presentation`

Thank you — questions welcome.